

TDT4186 - sample exam

1 Multiple-Choice Questions

1. Which of the following about process is **not correct**?

- A. Process is a running program.
- B. Every process has its own page table.
- C. Every process maintains a structure to store all opened files.
- D. Every process maintains a structure to store all locks it is using.**

2. What is the output of the following code snippet? (Assume that process creation succeeds, so no error check.)

```
ret = fork();
printf("A-");
if (ret > 0){
    wait();
    printf("B-");
}
else {
    printf("C-");
}
```

- A. A-B-C-
- B. A-C-B-
- C. A-A-B-C-
- D. A-A-C-B-**

3. The following task set is scheduled using STCF, what is the turnaround time of P1?

	Arrival Time	Execution Time
P1	0	30
P2	10	15
P3	20	10

- A. 30
- B. 45
- C. 50
- D. 55**

4. Which of the following scheduling algorithm maintains a circular queue?
- A. FIFO
 - B. SJF
 - C. STCF
 - D. RR**
5. Which of the following one is **correct** for CFS scheduling algorithm in Linux?
- A. The complexity of selecting the next scheduled task for CFS is $O(\log(n))$
 - B. CFS always schedules the task with the minimum *vruntime***
 - C. The design objective of CFS is to improve average response time
 - D. CFS deploys a B+ tree structure to store *vruntime* of all tasks
6. Which of the following one is **not correct**?
- A. Segmentation may cause external fragmentation.
 - B. Each segment has two registers, one to store the starting address of the segment and another to store the ending address of the segment.**
 - C. Paging may cause internal fragmentation.
 - D. Using TLB can reduce the number of memory accesses.
7. If a process has m different pages, the minimum number of page fault is
- A. m**
 - B. 0
 - C. 1
 - D. $m - 1$
8. When launching a process, an address space is created for this process. Which of the following one is **correct** about address space?
- A. For all OSes, address space only consists of three parts, i.e., code, heap, and stack
 - B. Heap and stack within address space can grow dynamically at runtime.**
 - C. Heap is used to store local variables.
 - D. Stack is used to store dynamically-allocated data structures.
9. Assuming that a process has 4 different pages, i.e., A, B, C, and D. Physical memory has 3 frames and the page access sequence is A-B-A-C-D. When accessing page D, which page will be replaced using LRU replacement policy? And how many page faults will the given page sequence have?
- A. Replace page A; 3 page faults
 - B. Replace page B; 4 page faults**

- C. Replace page C; 3 page faults
 - D. Replace page A; 4 page faults
10. Which of the following is **not** the requirement of solutions for critical-section problems?
- A. Mutual Exclusion
 - B. Non-preemption**
 - C. Progress
 - D. Bounded Waiting
11. Which of the following is **not** a condition for deadlock?
- A. Mutual Exclusion
 - B. Hold and Wait
 - C. Full Preemption**
 - D. Circular Wait
12. Which of the following statement is **true** regarding memory-mapped I/O?
- A. The CPU accesses the device memory much like it accesses main memory.**
 - B. The CPU uses separate architecture-specific instructions to access memory in the device.
 - C. Memory-mapped I/O can be used only with a polling-based device driver.
 - D. Memory-mapped I/O can be used only with an interrupt-driven device driver.
13. Which of the following concept is **not** a part of hard disk drives?
- A. Cylinder
 - B. Cell**
 - C. Sector
 - D. track
14. Processes 1, 2, and 3 read a file `test.txt` using `open()+read()`, then which of the following statement is **correct**?
- A. Processes cannot have same file descriptor number for `test.txt`.
 - B. If one process changes the offset of its file descriptor, all other file descriptors will update offsets accordingly.
 - C. All processes will have different inodes for file `test.txt`.
 - D. None of the above.**
15. Which of the following is **correct** for file systems?
- A. Superblock includes all information of free inodes blocks and data blocks.

- B. Directories are a specific-design data structure different from files.
- C. If an inode block is released, it can be reused for another file.**
- D. Bitmap can only be used for data blocks.
16. When reading a file `/root/bar.txt`, we need to access several parts of the on-disk structures to complete this operation
1. inode block of `root`
 2. inode block of `bar`
 3. data block of `root`
 4. data block of `bar`
- what is the correct access order?
- A. 1, 2, 3, 4
 - B. 1, 3, 4, 2
 - C. 1, 3, 2, 4**
 - D. 2, 1, 4, 3

2 Multi-Parts Questions

1. Given the following snippet,

```
int count = 0;
int i;
for (i=0; i<2; i++) {
    fork();
    count++;
}
printf("%d\n", count);
```

- (a) (1 point) How many processes will this snippet create at the end?

Solution: It will generate 4 processes.

`i=0`: It has the parent process and a newly created child process. The child duplicates the parent, so it has the same `i` and `count`.

`i=1`: Both the child and parent will call `fork` and thus create two child processes, so it will end up having 4 processes.

- (b) (1 point) What will the above snippet print out?

Solution: 2

2

2

2

Because every time a process calls `fork()`, it duplicates the parent and continues the execution just after `fork` call. So all processes have the same count value.

2. Paging deploys a per-process page table to store the information of translating page numbers to frame numbers for each process.

- (a) (1 point) Where are page tables stored? How many memory accesses does a virtual address need to point to the corresponding physical memory in page-based memory management without TLB?

Solution: Page tables are stored on the main memory. Every virtual address needs two memory references to point to its physical memory.

- (b) (1 point) What is the purpose of TLB? If a page entry is not found in TLB, i.e., page fault, how does OS handle a page fault?

Solution: TLB is translation lookaside buffer used to cache translations of virtual memory to physical memory. It is deployed to improve the performance of paging system. If a page fault occurs, OS first loads the required translation entry into TLB and then translate the virtual address to its physical address.

- (c) (2 points) What is LRU in page replacement? And how does the real system implement LRU?

Solution: LRU represents least-recently used policy. It is based on a conjecture that least recently used page is likely to not be used in future and thus it replaces such page to improve the performance of TLB. The real implementation of LRU is clock algorithm, where hardware support is needed. Instead of finding the exact page which has been used least recently, the algorithm uses a circular list and use bit to check whether a page is used or not. If the use bit is set to 1, it indicates the page has been used recently. It changes the use bit of the page to 0 and moves to the next page and until it finds a page with 0 use bit.

3. (a) (1 point) What is the hybrid lock? How does it work?

Solution: Spinlocks use a while loop to repeatedly evaluate locks' state, and it may waste CPU resources if waiting for a long period. A method to solve this wastage

is to yield the caller or deschedule itself if the lock is unavailable. However, both approaches have pros and cons. The hybrid lock just combines the spin and yield. When acquiring a lock, it will first spin for a short period and then yield the caller if the lock cannot be grabbed within that period.

- (b) (2 points) Can we implement a lock using semaphore? Assume `critical_sect()` is the critical section we are going to have mutual exclusion. Explain how to initialize semaphore and where should the two operations be added for the critical section?

Solution: A simple value assignment is allowed for semaphore initialization.

```
sem_t lock = 1;  
sem_wait(&lock);  
    critical_sect();  
sem_post(&lock);
```

4. Two threads access variable `x`, and their assembly code is as follows.

Thread 1	Thread 2
<code>mov R1, x // x -> R1;</code>	<code>mov R2, x // x -> R1;</code>
<code>inc R1 // R1 + 1 -> R1</code>	<code>inc R2 // R1 + 1 -> R1</code>
<code>mov x, R1 // R1 -> x</code>	<code>mov x, R2 // R2 -> x</code>

where `R1` and `R2` are two registers. Assume that `x = 2` and both threads only manipulate `x` once.

- (a) (1 point) How many different values can we have for `x` ?

Solution: 2 different values

- (b) (2 points) What are the values? How does the execution end up getting them?

Solution: If two threads execute to their completion without interrupt. The output value will be 4. If one thread is interrupted just before the third line, another thread will start with the old `x` value, 2. And then, regardless of which thread writes the value last, the final value of `x` will be 3.

5. Given an OS with a paging-based memory management, the system allows for **1GB** virtual address spaces for processes. The page size is **4KB**, and the system has **4 GB** of physical memory.

- (a) (1 point) How many physical frames does the system have?

Solution: $1\text{GB} = 2^{30}$; $4\text{GB} = 2^{32}$; $4\text{KB} = 2^{12}$;
frame number is $2^{32} \div 2^{12} = 2^{20}$

- (b) (1 point) How many bits does the virtual address have for virtual page number?

Solution: Since the maximum virtual address space covers 1GB and we have 4KB frame size, the number of frames is 2^{18} , it needs 18 bits in virtual address for the page number.

- (c) (1 point) If the size of page table entry (PTE) is 4 bytes and it has two-level page table, how many bits does it use to point to the first-level page directory?

Solution: Each page can store $2^{12} \div 4 = 2^{10}$ page table entries (PTEs). 10 bits are used to index the second-level page table and $18 - 10 = 8$ bits are used for the first-level page directory. (Other divisions like 10 bits for the PD Index and 8 for the PT Index or 9/9 both would also be possible)

- (d) (1 point) Consider a process references to 11 different virtual addresses and TLB can store 10 entries. The process repeatedly accesses the 11 virtual addresses in a fixed order, what will be the hit rate of TLB when using FIFO as the page replacement policy? What will be the hit rate of LRU?

Solution: The hit rate for FIFO will be 0, because after the 10th page, FIFO always replaces the page which will be accessed next time. LRU also has 0 hit rate.

6. Three threads access two shared semaphores as follows:

Thread 1	Thread 2	Thread 3
<pre>while (1){ sem_wait(U); printf("A"); sem_post(V); }</pre>	<pre>while (1){ sem_wait(V); printf("B"); sem_post(V); }</pre>	<pre>while (1){ sem_wait(V); printf("C"); }</pre>

where U is initialized to 3 and V is initialized to 0.

- (a) (1 point) If execution gets stuck, how many "A" will be printed out? Explain the reason.

Solution: 3. Because no thread can increment U's value and the initial value is 3, thread 1 will only execute 3 times.

- (b) (1 point) What is the minimal number of printed "B"? Explain the reason.

Solution: 0. Every time thread 1 increments V, and thread 3 decrements it. Thus, thread 2 always waits on V.

- (c) (1 point) Is "ACBABCABC" a possible output sequence? Explain the reason.

Solution: No. If A is printed, $V = 1$. Then, C is printed and V is decremented to 0. Thread 2 has to wait on V.